

CMPU4003 Advanced Databases

Phase 2 Report – Hybrid Data Warehouse (PostgreSQL + CouchDB)

Student Number:	C21394693
Student Name:	Stephen Thompson
Programme Code:	Tu858/4
Submission Date:	16/11/25

Table of Contents

1. Data Selection and Design Rationale.....	1
2. PostgreSQL Data Warehouse Design	2
2.1 Schema Design	2
2.2 ETL Process	4
2.3 Analytical Queries (non JSONB).....	4
2.4 JSONB Integration and Queries	8
2.5 Performance Comparison.....	Error! Bookmark not defined.
3. PostgreSQL Data Warehouse Optimization	9
3.2 Performance Evaluation	10
4. CouchDB Design and Analytics	14
4.1 Document Model.....	14
4.2 ETL / Population.....	15
4.3 Analytical Queries and Views	16
4.4 Comparison with Operational Relational DB.....	19
5. Comparative Discussion (PostgreSQL vs CouchDB)	20
6. Critical Reflection and Conclusion	21
7. Evidence of Deployment.....	21
8. References	23

1. Data Selection and Design Rationale

PostgreSQL:

The analytical goal for the PostgreSQL data warehouse is to support regional performance reporting. The design answers the analytical question of “Which regions are most popular by revenue and why?”. This is achieved by answering the sub questions of; what are our customers’ average spends?, which region is exhibiting the highest growth in the most recent month?

The following data was used to answer these questions:

Date – Transactions could be tracked over specific time periods to allow for granular examination.

Customer – Customer region and id was used to track unique customers in the region as well as to locate where spending was taking place

Merchant – Used to track the total sales of each merchant cross referenced with region to determine profitability per region.

Region – Region names could be associated with transactions so each transaction could be grouped according to the location of purchase

CouchDB:

CouchDB’s semi-structured layout was used to examine the customer trends from a more personal level such as what devices are being used per customer and what regions their transactions are taking place.

Rather than overall region metrics CouchDB allowed for each merchant to examine consumer trends on a smaller scale and examine not just aggregated numbers but individual metrics within the company’s sales.

Data used from original schema:

Customers – Linked to events for aggregation of customer events and regional activity

Events – Details customer interactions with merchants as well as medium metrics

Region – Links to customer events to display region textually

2. PostgreSQL Data Warehouse Design

2.1 Schema Design

The data warehouses schema design revolves around the fact table of region profitability, recording daily performance metrics. The granularity of the table is one record per region per day by date and region. Dimensions employed include date, region, customer, product and merchant for descriptive attributes associated with profitability. Region and merchant could be considered slow changing dimensions as only latest names and regions are stored. This schema supports analytics on profitability by allowing for fast querying on order density by region as well as scaling performance metrics.

Fact Table:

```
CREATE TABLE dw_lite_c21394693.fact_region_profitability(
  date_key INT REFERENCES dw_lite_c21394693.dim_date(date_key),
  region_key INT REFERENCES dw_lite_c21394693.dim_region(region_key),
  total_revenue NUMERIC(14,2),
  total_orders INT,
  total_customers INT,
  avg_spend_per_customer NUMERIC(12,2),
  unique_merchants_count INT,
  customers_to_merchants_ratio NUMERIC(8,2),
  performance_analysis jsonb NOT NULL DEFAULT '{}::jsonb
);
```

Dimension Tables:

```
CREATE TABLE dw_lite_c21394693.dim_date (
    date_key INT PRIMARY KEY,
    date_actual DATE,
    year INT,
    month INT,
    day INT
);

CREATE TABLE dw_lite_c21394693.dim_customer (
    customer_key BIGINT PRIMARY KEY,
    region TEXT,
    full_name TEXT,
    age_band TEXT
);

CREATE TABLE dw_lite_c21394693.dim_product (
```

```
product_key BIGINT PRIMARY KEY,  
category TEXT,  
merchant TEXT,  
price NUMERIC(12,2)  
);  
  
-- NEW DIMENSIONAL MODELLING ADDED BY ME  
CREATE TABLE dw_lite_c21394693.dim_merchant (  
    merchant_key BIGINT PRIMARY KEY,  
    merchant_name TEXT,  
    region TEXT  
);  
  
CREATE TABLE dw_lite_c21394693.dim_region (  
    region_key SMALLINT PRIMARY KEY,  
    region_code TEXT,  
    region_name TEXT  
);
```

2.2 ETL Process

The ETL extraction pulls from the relational sources of orders, customers, products, merchants and region. Using relevant joins, the fact table is enriched with dimensional attributes.

Transformations take place on the region by date level to compute transformative data like customer to merchant ratio to enrich data beyond relational source. JSONB functions are deployed for advanced analytical examination on growth analysis. We compute region profitability in totality and compare to recent sales figures to examine growth by region.

Unlogged and temp tables are employed to further optimize performance and compare to initial querying results for overall performance improvement when querying.

```
SELECT
  dd.date_key,
  dr.region_key,
  SUM(o.total_amount) AS total_revenue,
  COUNT(o.order_id) AS total_orders,
  COUNT(DISTINCT o.customer_id) AS total_customers,
  AVG(o.total_amount) AS avg_spend_per_customer,
  COUNT(DISTINCT o.merchant_id) AS unique_merchants_count,
  COUNT(DISTINCT o.customer_id) / COUNT(DISTINCT o.merchant_id) AS customers_to_merchants_ratio,
```

2.3 Analytical Queries (non JSONB)

Key analytical question answered by the data warehouse is “which region is most popular/ profitable?”.

This question is answered by our analytical queries, first being region-based profitability. Metrics such as region name and date are extracted as well as total revenue and total orders are aggregated from the fact table. These are then grouped by region name and date to examine daily transaction metrics to analyse trends within the database. These are ordered by date and sub ordered by total revenue to provide day by day rankings of each region within the database.

```
SELECT
  dr.region_name,
  dd.date_actual,
  SUM(f.total_revenue) AS total_revenue,
  SUM(f.total_orders) AS total_orders
FROM dw_lite_c21394693.fact_region_profitability f
JOIN dw_lite_c21394693.dim_region dr ON f.region_key = dr.region_key
JOIN dw_lite_c21394693.dim_date dd ON f.date_key = dd.date_key
GROUP BY dr.region_name, dd.date_actual
ORDER BY dd.date_actual, total_revenue DESC;
```

Growth analysis (using JSONB) is extracted from the data warehouse. From the jsonb object growth analysis a ratio of October sales to total sales is extracted as Growth Rate, total sales are examined as well as a growth analysis which returns a high moderate or poor score, which provides performance metrics at a glance when queried.

```
SELECT
    dr.region_name,
    f.performance_analysis->'growth_analysis' ->>'growth_rate' AS GrowthRate,
    f.performance_analysis->'total_sales' AS total_sales,
    f.performance_analysis->'growth_analysis' ->>'growth_analysis' AS GrowthAnalysis
FROM dw_lite_c21394693.fact_region_profitability f
JOIN dw_lite_c21394693.dim_region dr ON f.region_key = dr.region_key
WHERE
    f.performance_analysis -> 'growth_analysis' @> '{"growth_analysis": "High"}
```

Performance Comparison:

Below are the two queries used for performance comparison, one acting upon the relational source and one on the data warehouse

```
SELECT
    r.region_name as region,
    COUNT(DISTINCT o.order_id) AS total_orders,
    COUNT(DISTINCT c.customer_id) AS total_customers,
    COUNT(DISTINCT c.customer_id) / COUNT(DISTINCT o.order_id) AS customers_to_orders_ratio
FROM rel_src.orders o
JOIN rel_src.customers c ON o.customer_id = c.customer_id
JOIN rel_src.regions r ON c.region_id = r.region_id
GROUP BY r.region_name
ORDER BY customers_to_orders_ratio DESC;

SELECT
    dr.region_name,
    SUM(f.total_orders) AS total_orders,
    SUM(f.total_customers) AS total_customers,
    SUM(f.total_customers) / SUM(f.total_orders) AS customers_to_orders_ratio
FROM dw_lite_c21394693.fact_region_profitability f
JOIN dw_lite_c21394693.dim_region dr ON f.region_key = dr.region_key
GROUP BY dr.region_name
ORDER BY customers_to_orders_ratio DESC;
```

EXPLAIN ANALYSE output from relational source:

```
"Sort (cost=4892.56..4892.57 rows=4 width=35) (actual time=52.512..52.514 rows=4
loops=1)"
"  Sort Key: ((count(DISTINCT c.customer_id) / count(DISTINCT o.order_id))) DESC"
"  Sort Method: quicksort  Memory: 25kB"
"  -> GroupAggregate (cost=4492.47..4892.52 rows=4 width=35) (actual
time=48.026..52.507 rows=4 loops=1)"
"    Group Key: r.region_name"
"    -> Sort (cost=4492.47..4592.47 rows=40000 width=27) (actual
time=45.290..46.692 rows=40000 loops=1)"
"      Sort Key: r.region_name, o.order_id"
"      Sort Method: quicksort  Memory: 3163kB"
"      -> Hash Join (cost=183.59..1434.93 rows=40000 width=27) (actual
time=1.164..17.228 rows=40000 loops=1)"
"        Hash Cond: (c.region_id = r.region_id)"
"        -> Hash Join (cost=182.50..1217.59 rows=40000 width=18) (actual
time=1.147..11.470 rows=40000 loops=1)"
"          Hash Cond: (o.customer_id = c.customer_id)"
"          -> Seq Scan on orders o (cost=0.00..930.00 rows=40000 width=16)
(actual time=0.002..2.892 rows=40000 loops=1)"
"            -> Hash (cost=120.00..120.00 rows=5000 width=10) (actual
time=1.138..1.138 rows=5000 loops=1)"
"              Buckets: 8192  Batches: 1  Memory Usage: 270kB"
"            -> Seq Scan on customers c (cost=0.00..120.00 rows=5000
width=10) (actual time=0.002..0.519 rows=5000 loops=1)"
"              -> Hash (cost=1.04..1.04 rows=4 width=13) (actual time=0.013..0.013
rows=4 loops=1)"
"                Buckets: 1024  Batches: 1  Memory Usage: 9kB"
"              -> Seq Scan on regions r (cost=0.00..1.04 rows=4 width=13) (actual
time=0.009..0.009 rows=4 loops=1)"
"Planning Time: 0.282 ms"
"Execution Time: 52.666 ms"
```

Explain output from data warehouse

```
"Sort (cost=29.32..29.82 rows=200 width=56) (actual time=0.258..0.260 rows=4
loops=1)"
"  Sort Key: ((sum(f.total_customers) / sum(f.total_orders))) DESC"
"  Sort Method: quicksort  Memory: 25kB"
"  -> HashAggregate (cost=19.18..21.68 rows=200 width=56) (actual
time=0.250..0.253 rows=4 loops=1)"
"    Group Key: dr.region_name"
"    Batches: 1  Memory Usage: 40kB"
"    -> Nested Loop (cost=0.16..17.35 rows=244 width=40) (actual
time=0.032..0.185 rows=244 loops=1)"
"      -> Seq Scan on fact_region_profitability f (cost=0.00..9.44 rows=244
width=12) (actual time=0.011..0.044 rows=244 loops=1)"
"        -> Memoize (cost=0.16..0.37 rows=1 width=34) (actual time=0.000..0.000
rows=1 loops=244)"
"          Cache Key: f.region_key"
"          Cache Mode: logical"
"          Hits: 240  Misses: 4  Evictions: 0  Overflows: 0  Memory Usage: 1kB"
"          -> Index Scan using dim_region_pkey on dim_region dr (cost=0.15..0.36
rows=1 width=34) (actual time=0.003..0.003 rows=1 loops=4)"
"            Index Cond: (region_key = f.region_key)"
"Planning Time: 0.129 ms"
"Execution Time: 0.297 ms"
```

As can be seen from the analysis of the queries, execution time for the warehouse is significantly faster than the operational source, can handle more concurrent operations without strain, requires less joins, and uses significantly less memory.

2.4 JSONB Integration and Queries

The semi-structured data stored in the DW is an aggregation of performance metrics of all time and just October sales. This is complimented by a growth rate comparing these metrics to establish a ratio of growth compared to these two metrics, as well as a growth analysis of these metrics to provide a single word summary of the data

```
SELECT
  dr.region_name,
  f.performance_analysis->'growth_analysis' ->>'growth_rate' AS GrowthRate,
  f.performance_analysis->'total_sales' AS total_sales,
  f.performance_analysis->'growth_analysis' ->>'growth_analysis' AS GrowthAnalysis
FROM dw_lite_c21394693.fact_region_profitability f
JOIN dw_lite_c21394693.dim_region dr ON f.region_key = dr.region_key
WHERE
  f.performance_analysis -> 'growth_analysis' @> '{"growth_analysis": "High"}'
```

3. PostgreSQL Data Warehouse Optimization

3.1 Optimization Techniques

UNLOGGED tables:

By altering the table to unlogged we increase efficiency of querying by removing the requirements to write a log before execution. The query can immediately output results

```
ALTER TABLE dw_lite_c21394693.fact_region_profitability SET UNLOGGED;
```

TEMP tables:

Exist for the lifespan of the session to speed up large aggregations as data does not need to be recalculated per query, it is instead stored in TEMP tables to increase querying efficiency.

```
CREATE TEMP TABLE temp_fact_region_revenue AS SELECT * FROM dw_lite_c21394693.fact_region_profitability;
```

Pg_prewarm:

Loads relational data into the system buffer cache of the PostgreSQL buffer cache. Using pg_prewarm we can load this data manually to increase querying efficiency by having data readily available for querying

```
SELECT pg_prewarm('dw_lite_c21394693.fact_region_profitability');
```

Indexing:

Indexing allows for the storage of JSON data in binary format, increases efficiency by allowing for specific JSONB data within JSON documents to be located quickly using individual keys and values.

3.2 Performance Evaluation

UNLOGGED

Pre-Optimisation	Post-Optimisation
"Sort (cost=36.24..36.85 rows=244 width=76) (actual time=0.670..0.683 rows=244 loops=1)" " Sort Key: dd.date_actual, (sum(f.total_revenue)) DESC" " Sort Method: quicksort Memory: 37kB" " -> HashAggregate (cost=23.51..26.56 rows=244 width=76) (actual time=0.512..0.569 rows=244 loops=1)" " Group Key: dd.date_actual, dr.region_name" " Batches: 1 Memory Usage: 157kB" " -> Hash Join (cost=3.21..21.07 rows=244 width=48) (actual time=0.230..0.400 rows=244 loops=1)" " Hash Cond: (f.date_key = dd.date_key)" " -> Nested Loop (cost=0.16..17.35 rows=244 width=48) (actual time=0.023..0.155 rows=244 loops=1)" " -> Seq Scan on fact_region_profitability f (cost=0.00..9.44 rows=244 width=20) (actual time=0.007..0.033 rows=244 loops=1)" " -> Memoize (cost=0.16..0.37 rows=1 width=34) (actual time=0.000..0.000 rows=1 loops=244)" " Cache Key: f.region_key" " Cache Mode: logical" " Hits: 240 Misses: 4 Evictions: 0 Overflows: 0 Memory Usage: 1kB" " -> Index Scan using dim_region_pkey on dim_region dr (cost=0.15..0.36 rows=1 width=34) (actual time=0.003..0.003 rows=1 loops=4)" " Index Cond: (region_key = f.region_key)"	"Sort (cost=53.68..54.63 rows=380 width=76) (actual time=0.322..0.331 rows=244 loops=1)" " Sort Key: dd.date_actual, (sum(f.total_revenue)) DESC" " Sort Method: quicksort Memory: 37kB" " -> HashAggregate (cost=32.65..37.40 rows=380 width=76) (actual time=0.220..0.260 rows=244 loops=1)" " Group Key: dd.date_actual, dr.region_name" " Batches: 1 Memory Usage: 157kB" " -> Hash Join (cost=3.21..28.85 rows=380 width=48) (actual time=0.036..0.150 rows=244 loops=1)" " Hash Cond: (f.date_key = dd.date_key)" " -> Nested Loop (cost=0.16..24.76 rows=380 width=48) (actual time=0.015..0.103 rows=244 loops=1)" " -> Seq Scan on fact_region_profitability f (cost=0.00..13.80 rows=380 width=20) (actual time=0.006..0.028 rows=244 loops=1)" " -> Memoize (cost=0.16..0.30 rows=1 width=34) (actual time=0.000..0.000 rows=1 loops=244)" " Cache Key: f.region_key" " Cache Mode: logical" " Hits: 240 Misses: 4 Evictions: 0 Overflows: 0 Memory Usage: 1kB" " -> Index Scan using dim_region_pkey on dim_region dr (cost=0.15..0.29 rows=1 width=34) (actual time=0.002..0.002 rows=1 loops=4)" " Index Cond: (region_key = f.region_key)"

" -> Hash (cost=1.91..1.91 rows=91 width=8) (actual time=0.202..0.203 rows=91 loops=1)" " Buckets: 1024 Batches: 1 Memory Usage: 12kB" " -> Seq Scan on dim_date dd (cost=0.00..1.91 rows=91 width=8) (actual time=0.183..0.191 rows=91 loops=1)" "Planning Time: 2.890 ms" "Execution Time: 0.748 ms"	" -> Hash (cost=1.91..1.91 rows=91 width=8) (actual time=0.016..0.016 rows=91 loops=1)" " Buckets: 1024 Batches: 1 Memory Usage: 12kB" " -> Seq Scan on dim_date dd (cost=0.00..1.91 rows=91 width=8) (actual time=0.003..0.008 rows=91 loops=1)" "Planning Time: 0.872 ms" "Execution Time: 0.371 ms"
---	---

TEMP TABLE

Pre-Optimisation	Post-Optimisation
"Sort (cost=36.24..36.85 rows=244 width=76) (actual time=0.670..0.683 rows=244 loops=1)" " Sort Key: dd.date_actual, (sum(f.total_revenue)) DESC" " Sort Method: quicksort Memory: 37kB" " -> HashAggregate (cost=23.51..26.56 rows=244 width=76) (actual time=0.512..0.569 rows=244 loops=1)" " Group Key: dd.date_actual, dr.region_name" " Batches: 1 Memory Usage: 157kB" " -> Hash Join (cost=3.21..21.07 rows=244 width=48) (actual time=0.230..0.400 rows=244 loops=1)" " Hash Cond: (f.date_key = dd.date_key)" " -> Nested Loop (cost=0.16..17.35 rows=244 width=48) (actual time=0.023..0.155 rows=244 loops=1)" " -> Seq Scan on fact_region_profitability f (cost=0.00..9.44 rows=244 width=20) (actual time=0.007..0.033 rows=244 loops=1)" " -> Memoize (cost=0.16..0.37 rows=1 width=34) (actual time=0.000..0.000 rows=1 loops=244)" " Cache Key: f.region_key" " Cache Mode: logical" " Hits: 240 Misses: 4 Evictions: 0 Overflows: 0 Memory Usage: 1kB" " -> Index Scan using dim_region_pkey on dim_region dr (cost=0.15..0.36 rows=1 width=34) (actual time=0.003..0.003 rows=1 loops=4)" " Index Cond: (region_key = f.region_key)"	"Sort (cost=69.31..70.03 rows=287 width=76) (actual time=0.513..0.529 rows=244 loops=1)" " Sort Key: dd.date_actual, (sum(f.total_revenue)) DESC" " Sort Method: quicksort Memory: 37kB" " -> HashAggregate (cost=54.01..57.60 rows=287 width=76) (actual time=0.339..0.406 rows=244 loops=1)" " Group Key: dd.date_actual, dr.region_name" " Batches: 1 Memory Usage: 157kB" " -> Hash Join (cost=32.40..51.14 rows=287 width=58) (actual time=0.087..0.215 rows=244 loops=1)" " Hash Cond: (f.region_key = dr.region_key)" " -> Hash Join (cost=3.05..21.03 rows=287 width=30) (actual time=0.057..0.144 rows=244 loops=1)" " Hash Cond: (f.date_key = dd.date_key)" " -> Seq Scan on temp_fact_region_revenue f (cost=0.00..16.30 rows=630 width=30) (actual time=0.011..0.045 rows=244 loops=1)" " -> Hash (cost=1.91..1.91 rows=91 width=8) (actual time=0.037..0.037 rows=91 loops=1)" " Buckets: 1024 Batches: 1 Memory Usage: 12kB" " -> Seq Scan on dim_date dd (cost=0.00..1.91 rows=91 width=8) (actual time=0.007..0.015 rows=91 loops=1)" " -> Hash (cost=18.60..18.60 rows=860 width=34) (actual time=0.020..0.020 rows=4 loops=1)"

" -> Hash (cost=1.91..1.91 rows=91 width=8) (actual time=0.202..0.203 rows=91 loops=1)" " Buckets: 1024 Batches: 1 Memory Usage: 12kB" " -> Seq Scan on dim_date dd (cost=0.00..1.91 rows=91 width=8) (actual time=0.183..0.191 rows=91 loops=1)" "Planning Time: 2.890 ms" "Execution Time: 0.748 ms"	" Buckets: 1024 Batches: 1 Memory Usage: 9kB" " -> Seq Scan on dim_region dr (cost=0.00..18.60 rows=860 width=34) (actual time=0.015..0.016 rows=4 loops=1)" "Planning Time: 1.834 ms" "Execution Time: 0.604 ms"
---	--

4. CouchDB Design and Analytics

4.1 Document Model

Embedded/ linked structure

The CouchDB JSON document model uses embedded and linked structures. For example, customer events are embedded in each customer document and linked via foreign keys such as region. Embedding data allows for the efficient retrieval of customer histories and activities, which supports analytics such as behaviours, trends and performance analysis.

The data complements the initial analytical goal of region performance by gaining a deeper understanding of what mediums customers are using to make purchases to better target which infrastructure requires the most attention per region.

```
{
  "_id": "customer_1",
  "_rev": "1-e8a1814cb683cc8bb0d4f5ddb231bb1d",
  "customer_id": 1,
  "full_name": "Customer 1",
  "region": "United States",
  "events": [
    {
      "event_id": 1,
      "session_id": "eb1eabbe-d9ff-4278-aaa8-db6f15edfe20",
      "event_ts": "2025-10-18T18:21:58.33356+00:00",
      "event_type": "page_view",
      "payload": {
        "geo": {
          "lat": 57.575818433229585,
          "lon": -10.902960169561407
        },
        "ref": "search",
        "page": "/home",
        "device": "tablet",
        "session_duration_sec": 144
      }
    }
  ]
}
```

4.2 ETL / Population Process

Generating the data used in the CouchDB document store is done by querying the relational source using `json_agg` and building the JSON objects using an embedded structure. Each entry per customer contains customer details as well as an embedded object of customer activity for swift retrieval and analysis.

The bulk import was completed using `curl` to inject the thousands of customer records using one simple command.

The data retrieved from the relational query was saved locally to a `.json` file as an array of customer objects with embedded activity and then a bulk upload command was used to segment each customer into its own entry allowing for querying per customer and activity.

4.3 Analytical Queries and Views

customersByRegion:

Examines where customers are located within the database and returns the customer id followed by the region they are in.

Reduces to a count by region

Example of output:

customer_1000	Europe	1
customer_1001	Europe	1
customer_1004	Europe	1
customer_1005	Europe	1
customer_1012	Europe	1
customer_1014	Europe	1
customer_102	Europe	1
customer_1021	Europe	1
customer_1022	Europe	1
customer_1026	Europe	1
customer_1032	Europe	1
customer_1036	Europe	1
customer_1039	Europe	1
customer_104	Europe	1
customer_1042	Europe	1

Reduce:

<

customer_events_...

:

All Documents

Run A Query with Mango

Permissions

Changes

Design Documents

▼ customersByregion

Metadata

▼ Views

sortByregion

► devicesUsed

Document ID

Options

{ } JSON

Create Document

Metadata

key	value
Europe	1731
Ireland	811
United Kingdom	1671
United States	787

```
{  
  "_id": "_design/customersByregion",  
  "_rev": "1-fb3a476b7803fa895611bf1fdd948462",  
  "views": {  
    "sortByregion": {  
      "map": "function(doc){emit (doc.region, 1);}",  
      "reduce": "_count"  
    }  
  }  
}
```

devicesUsed:

Returns a list of customers by id and the device they used (in this example tablet) per event

Reduces to a count of each event that took place on a tablet

Example of output:

customer_1	tablet	1
customer_1	tablet	1
customer_10	tablet	1
customer_1000	tablet	1
customer_1000	tablet	1
customer_1001	tablet	1
customer_1002	tablet	1
customer_1002	tablet	1
customer_1003	tablet	1
customer_1005	tablet	1
customer_1006	tablet	1
customer_1007	tablet	1
customer_1009	tablet	1
customer_101	tablet	1

Reduce:

<

customer_events_...

⋮

All Documents

Run A Query with Mango

Permissions

Changes

Design Documents

customersByregion

Metadata

Views

sortByregion

devicesUsed

Metadata

Views

devicesUsed

Document ID

Options

{ } JSON

📖

🔔

Metadata

key

value

tablet

4999

Create Document

```
{
  "_id": "_design/devicesUsed",
  "_rev": "2-a907ca238c20b83672df4127673ce4a7",
  "views": {
    "devicesUsed": {
      "map": "function(dev){dev.events.forEach(function(events){if(events.payload &&
events.payload.device == 'tablet'){emit('tablet', 1); }});}",
      "reduce": "_count"
    }
  }
}
```

4.4 Comparison with Operational Relational DB

5. Comparative Discussion (PostgreSQL vs CouchDB)

Query Capabilities:

PostgreSQL being a relational database supports complex joins and analytical queries. JSONB integration allows for a mix of structured and semi structured results for large data sets suited to tables containing large numerical data requiring aggregation and analysis.

CouchDB being document-oriented and optimized for JSON storage primarily focuses in MapReduce and Mango queries, these are optimized for retrieving hierarchical structured or nested data.

Performance Scalability:

PostgreSQL

Performs well under the conditions of large data aggregation ie performing calculations on tables with thousands to millions of rows for analytical data. Efficient under large datasets especially when using optimisations such as UNLOGGED tables, TEMP tables and pg_prewarm to produce results.

CouchDB

Designed as a document store CouchDB is most efficient in retrieval of JSON elements without a rigid schema format. JSON ingestion is efficient and optimized for querying but struggles in efficient aggregation due to its lack of inherent datatypes for aggregation

Schema Rigidity vs Flexibility:

PostgreSQL

PostgreSQL operates under strict schema rigidity, which means table creation and updating must adhere to predetermined datatypes, rules and modelling. This can hinder development, especially with document-oriented data as each record must adhere to rigid structures.

This allows for incredibly efficient aggregation but falters with varying data that does not follow strict patterns.

CouchDB

CouchDB being schema-less allows for flexible document definitions and evolving documents meaning data is less fixed in its definition. This is ideal for varying data such as customer events as the inclusion of additional events does not require the restructuring of tables or the creation of new tables using complex joins.

The nested nature of CouchDB documents allows for new data to be inserted without the use of complex joins or updates.

CouchDB Compliment

CouchDB compliments the DW as they can work in tandem to enrich the analytical goal of the data warehouse. With the chaotic nature of customer records within our relational source, CouchDBs schema-less design allows for different facets of the analytical goal to be examined that a rigidly designed schema would struggle to

produce. The data warehouse thrives in aggregation and CouchDB compliments this by allowing for textual answers to the same analytical question where numbers may not suffice.

The hybrid strengths of the data warehouse and CouchDB working in tandem complement each other by allowing for multiple angles of attack to answer the same analytical question.

Answers formed using these hybrid strengths are well rounded and multi-faceted.

6. Critical Reflection and Conclusion

I believe the decisions made throughout the creation of this complementary warehouse and CouchDB database allows for a complete picture to be painted of the analytical goals.

The tradeoffs seen in the decision of the data split are apparent.

Allowing the data warehouse to handle aggregation of sales numbers appears to be common sense but the results can lack depth without context. Although it is simple to examine the numeric qualities of each region without these numbers being put into context of where the sales appear from may be lacking in depth.

In having the flexible results generated by Couch we gain a more well rounded view of why the numbers appear in a certain fashion, we have a clear picture of how the data came to be by examining user trends that created these numbers.

This created a durable structure in our warehouse with concrete analytical numbers and CouchDB takes over the more flexible data to produce efficient results PostgreSQL would struggle with.

7. Evidence of Deployment

Docker:

☐	>		couch	-	-	-	0.43%	1 day ago				
☐	>		database	-	-	-	0.07%	1 day ago				

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
77bd6bdc2dad	couchdb:latest	"tini -- /docker-ent..."	5 weeks ago	Up 27 hours	0.0.0.0:5984->5984/tcp, [::]:5984->5984/tcp	couchdb
177b1656a80b	dpag/pgadmin4	"/entrypoint.sh"	5 weeks ago	Up 27 hours	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	dev-pgadmin
a881edfc3ee9	postgres:17	"docker-entrypoint.s..."	5 weeks ago	Up 27 hours	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	dev-postgres

PostgreSQL:

CA2/steuser@test

Query Query History

```
1 SELECT
2   dr.region_name,
3   dd.date_actual,
4   SUM(f.total_revenue) AS total_revenue,
5   SUM(f.total_orders) AS total_orders
6 FROM dw_lite_c21394693.fact_region_profitability f
7 JOIN dw_lite_c21394693.dim_region dr ON f.region_key = dr.region_key
8 JOIN dw_lite_c21394693.dim_date dd ON f.date_key = dd.date_key
9 GROUP BY dr.region_name, dd.date_actual
10 ORDER BY dd.date_actual, total_revenue DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 244 Page No: 1 of 1

	region_name text	date_actual date	total_revenue numeric	total_orders bigint
1	Europe	2025-09-11	1258.55	12
2	United Kingdo...	2025-09-11	994.49	11
3	United States	2025-09-11	714.85	6
4	Ireland	2025-09-11	106.28	2
5	Europe	2025-09-12	26743.35	248
6	United Kingdo...	2025-09-12	24232.98	216
7	United States	2025-09-12	12492.93	104
8	Ireland	2025-09-12	11625.59	110
9	Europe	2025-09-13	26588.13	237
10	United Kingdo...	2025-09-13	21295.51	200
11	Ireland	2025-09-13	13948.25	124
12	United States	2025-09-13	11125.85	101
13	Europe	2025-09-14	27240.92	246

CouchDB:

customer_events_c21394693 4.4 MB 5002 No

customer_events_...

Document ID Options {} JSON

All Documents Run A Query with Mango Permissions Changes Design Documents customersByregion devicesUsed

Create Document

id	key	value
._design/customersByregion	._design/customersByregion	{ "rev": "1-fb3a476b7803fa895611bf1fd9...
._design/devicesUsed	._design/devicesUsed	{ "rev": "2-a907ca238c20b83672df412767...
customer_1	customer_1	{ "rev": "1-e8a1814cb683cc8bb0d4f5ddb2...
customer_10	customer_10	{ "rev": "1-ccfe3bbde85ecf106242b8ba39...
customer_100	customer_100	{ "rev": "1-219768dfc73a3002dd4c02e498...
customer_1000	customer_1000	{ "rev": "1-4225e9f63a3f2105a4a27eca111...
customer_1001	customer_1001	{ "rev": "1-c0df2fc3c73f4add995f6bc2fa5d...
customer_1002	customer_1002	{ "rev": "1-f5adf5fb3efe38cf66268cb68552...
customer_1003	customer_1003	{ "rev": "1-574b9497a89108c1c7358c5c4b...
customer_1004	customer_1004	{ "rev": "1-bbf9913e7b1376e1e9b50f3b2a...
customer_1005	customer_1005	{ "rev": "1-1f2a009ec0739965e32103a372...
customer_1006	customer_1006	{ "rev": "1-873cf2caf94dc6349ce4ec60f9...
customer_1007	customer_1007	{ "rev": "1-748610bce1cd56898e6730f9d...
customer_1008	customer_1008	{ "rev": "1-16bb445e310323a6dbad96a59f...
customer_1009	customer_1009	{ "rev": "1-b4f899722a950d117bd3804130...

8. References